

Attention: motivation

Let us consider a task familiar to all of us who have taken standardized exams: Question Answering.

Input consists of two pieces of text

- a paragraph (called the *Context*)
- a Question whose answer can be found in the paragraph

Output is a piece of text that answers the question.

$\mathbf{x} =$

{ Context: The FRE Dept offers many Spring classes. The students are gr
:
Professor Perry taught them Machine Learning. The studen
:
Professor Blecherman led a class in ...
:
Question: What did Professor Perry do ?

$\mathbf{y} =$ Answer: He taught them Machine Learning

This is an example of a *Sequence to Sequence* task

- briefly encountered when we defined RNN's

Language translation is another common Sequence to Sequence task.

Let us hypothesize how a model might learn to solve this task

- it is only an hypothesis: we don't really know

The model might have generalized

- from seeing many training examples of disparate questions and their answers
- that there is a parameterized *template* for both the Question and the Answer

Question Template

What did Professor <PROPER NOUN> teach in the Spring?

Answer Template:

<PRONOUN> <VERB> <INDIRECT OBJECT> <OBJECT>

where <PROPER NOUN>, <PRONOUN>, <VERB>, etc. are *pattern place-holders* parameters.

By using a parameterized template, the model captures

- commonality
- in many different types of questions

In order to produce the answer the model needs to

- Generate the tokens of the Answer Template in order
- Substituting in concrete values for the place-holders
 - by performing a Lookup in the Context in order to obtain these values

We will examine how the Lookup might be performed

- first: by using mechanisms that we have already studied
- subsequently: via a new mechanism called *Attention*

Using an RNN without Attention

Encoder-Decoder architecture: review

For the Question Answering task

- both the Input and Output are sequences
- thus, the task is a Sequence to Sequence task
 - just like: Language Translation

We learned that Recurrent architectures are best-suited for processing sequences.

These architectures

- operate in a "loop"
 - processing one Input or Output token at a time
- utilize **memory** (latent state)
 - necessary because Input/Output sequence lengths are unbounded
 - after processing the token at position
 - the latent state is finite representation of the prefix of the sequence of length

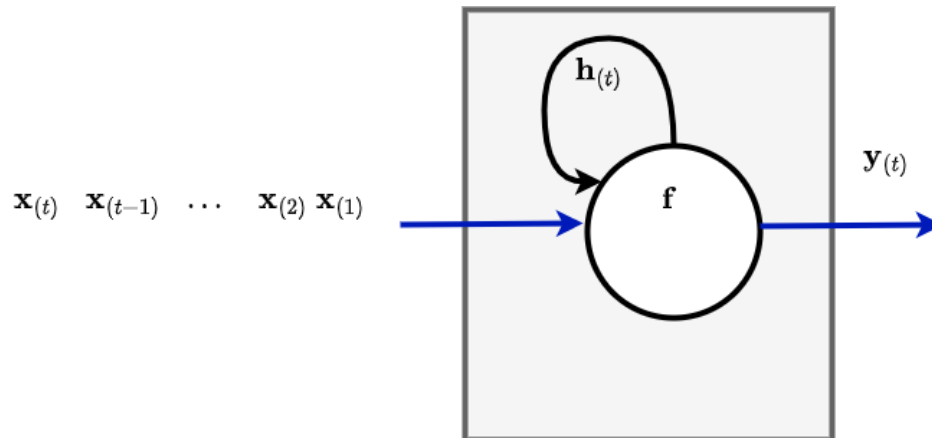
Loop architecture

- Uses a "latent state" that is updated with each element of the sequence, then predict the output

$\text{pr } \mathbf{h}_{tp} | \mathbf{x}_{tp}, \mathbf{h}_{(-1)}$ latent variable $\mathbf{h}_{tp}$ encodes $[\mathbf{x}_{(1)} \dots \mathbf{x}_{tp}]$

$\text{pr } \hat{\mathbf{y}}_{tp} | \mathbf{h}_{tp}$ prediction contingent on latent variable

Loop with latent state



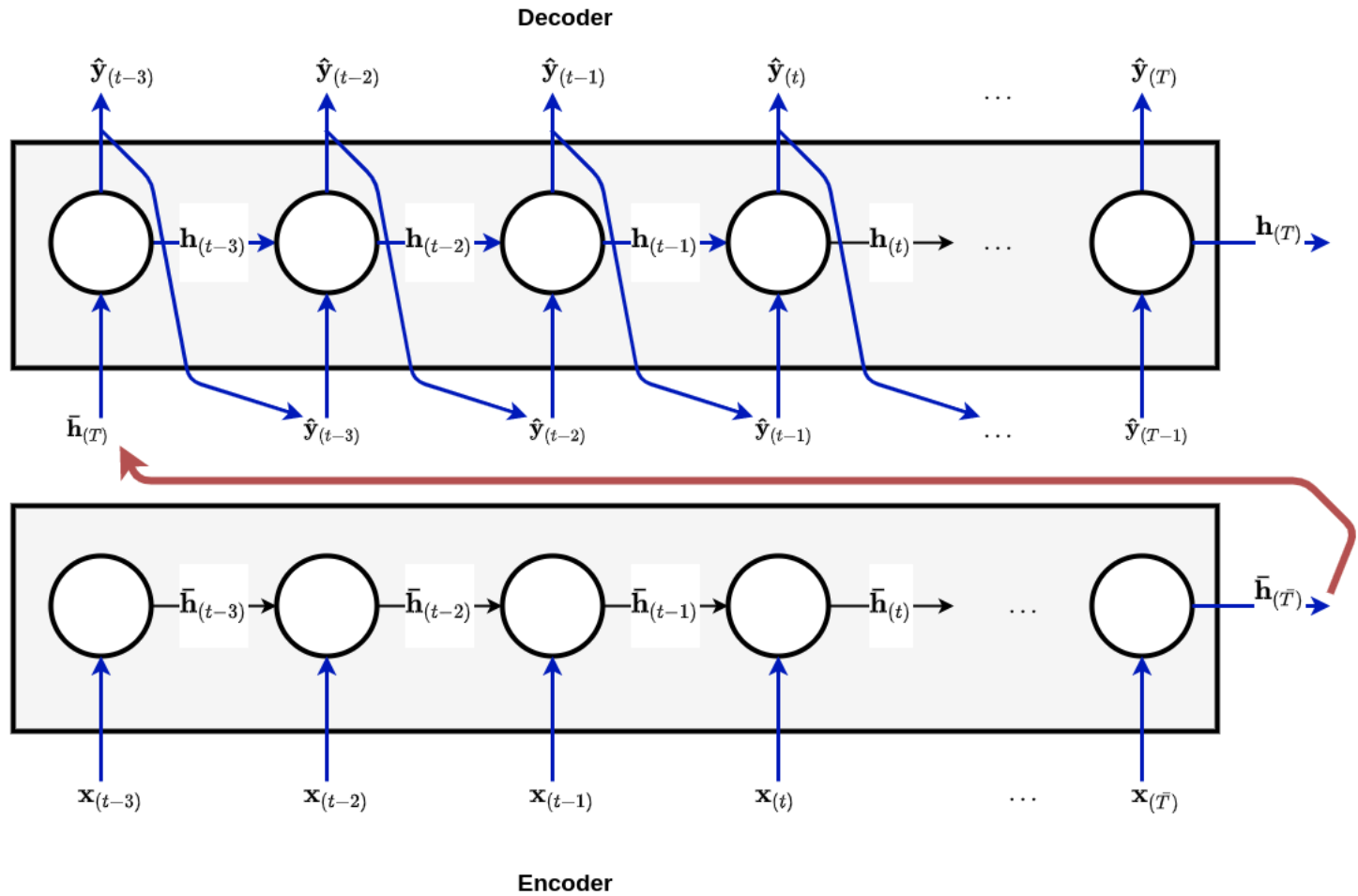
A common architecture for Sequence to Sequence tasks is the Encoder-Decoder:

- The Encoder is an RNN
 - Acts on input sequence $[\mathbf{x}_{(1)} \dots \mathbf{x}_{(\bar{T})}]$
 - Producing a sequence of latent states $[\bar{\mathbf{h}}_{(1)}, \dots, \bar{\mathbf{h}}_{(\bar{T})}]$
 - latent state $\bar{\mathbf{h}}_{tt}$ is a summary of $[\mathbf{x}_{(1)} \dots \mathbf{x}_{tp}]$

The Decoder

- Acts on the *final* Encoder latent state $\bar{\mathbf{h}}_{(T)}$
 - which summarizes the entire input sequence $\mathbf{x}$
 - Producing a sequence of latent states $[\mathbf{h}_{(1)}, \dots, \mathbf{h}_{(T)}]$
 - latent state $\mathbf{h}_{tp}$ is response for generating output token $\hat{\mathbf{y}}_{tp}$
 - Thus outputting a sequence $[\hat{\mathbf{y}}_{(1)}, \dots, \hat{\mathbf{y}}_{(T)}]$

RNN Encoder/Decoder



Auto-regressive Decoder

The Decoder usually works in an *Auto-Regressive* manner

- output $\hat{\mathbf{y}}_{t_p}$ of step
- becomes part of the input $\hat{\mathbf{y}}_{(1:t)}$ of step +1
- see illustration above

The final output sequence is thus constructed

- single element at a time
- with a sequential dependency on all prior elements of the output sequence

Decoder output $\hat{y}_{tp}$

The simplest RNN (corresponding to our diagrams) use the latent state h_{tp} as the output $\hat{y}_{tp}$

$$\hat{y}_{tp} = h_{tp}$$

It is easy to add another NN to transform h_{tp} into a $\hat{y}_{tp}$ that is different.

- We can add a NN to the Decoder RNN that implements a function D that transforms the latent state into an output.

$$\hat{y}_{tp} = D(h_{tp})$$

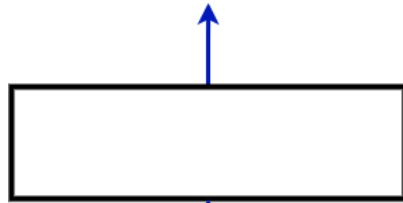
For clarity: we will omit this additional NN from our diagrams until it becomes necessary

Here is what the additional NN looks like:

Decoder output transformation: No attention

Decoder

$$\hat{\mathbf{y}}_{(1)}, \hat{\mathbf{y}}_{(2)}, \dots, \hat{\mathbf{y}}_{(T)}$$



$$\mathbf{h}_{(t)}$$

How does the Decoder perform Lookup (without Attention) ?

Suppose the Decoder has already output

$$\hat{\mathbf{y}}_{([1:3])} = \text{He taught them}$$

It must subsequently output

$$\hat{\mathbf{y}}_{([4:5])} = \text{Machine Learning}$$

In order to do this

- it must Lookup "Machine Learning" in the Context, resulting in

$$D(\backslash \mathbf{h}_{(4)}) = \text{Machine}$$

$$D(\backslash \mathbf{h}_{(5)}) = \text{Learning}$$

But D is conditioned on the single input $\backslash \mathbf{h}_{tp}$.

Thus, in order for $D(\backslash \mathbf{h}_{(4)})$ to be equal to "Machine"

- this information must somehow be encoded in $\backslash \mathbf{h}_{(4)}$

How did it get there ?

All "knowledge" from the Context must be transferred from Encoder to Decoder

- through final Encoder state $\bar{\mathbf{h}}_{(\bar{T})}$
- which in turn was encoded in all Encoder states $\bar{\mathbf{h}}_{(-')}$ for $' \geq \bar{p}$
 - where $\bar{p}$ is the position within sequence $\mathbf{x}$ of the word "Machine"

We can hypothesize that the final Encoder latent state $\bar{\mathbf{h}}_{T^*}$

- encodes a Dictionary (key/value pairs)
- mapping Place-holder names to Concrete values
- the dictionary is built incrementally by prior latent states of the Encoder

Answering questions using Attention

Input Tokens: Professor Perry taught them Machine Learning [CLS]

$$\bar{\mathbf{h}}_{(1)}, \dots, \bar{\mathbf{h}}_{(T^*)} = \left\{ \begin{array}{l} \text{Subject: Professor Perry} \\ \text{Pronoun: he} \\ \text{Object: Machine Learning} \\ \text{Indirect Object: them} \\ \text{Verb: taught} \end{array} \right\}$$

This dictionary is passed to the Decoder via the single connection from Encoder to Decoder

- and must be carried forward by the Decoder
- through Decoder states $[\backslash \mathbf{h}_{(1)}, \dots, \backslash \mathbf{h}_{(4)}]$
- in order to make the dictionary available to subsequent latent states of the Decoder

We further hypothesize that the Decoder

- performs Lookups
- by using the Decoder latent state $\mathbf{h}_{tp}$
 - as a *query* that matches against the keys of the Dictionary
 - in order to obtain the Concrete value required to produce output token at position

$$\hat{\mathbf{y}}_{tp} = D(\mathbf{h}_{tp})$$

Query performing a Lookup in the Dictionary

Question: What did Professor Perry do ?

Answer template: <PRONOUN> <VERB> <INDIRECT OBJECT> <OBJECT> [CLS]

Queries:

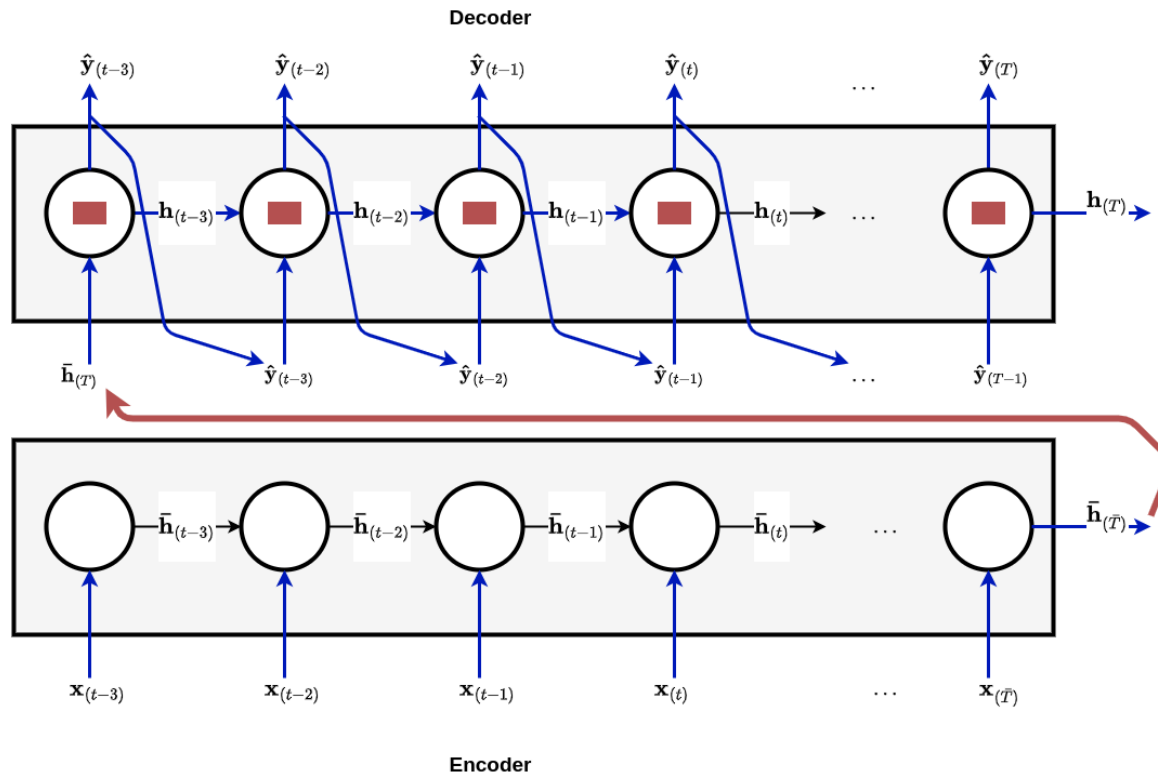
$$\mathbf{h}_{(\text{Pronoun})} = \{ \text{Pronoun: ?} \}$$

$$\mathbf{h}_{(\text{Verb})} = \{ \text{Verb: ?} \}$$

Answer: He taught them Machine Learning [CLS]

Here is a picture describing this hypothetical functioning.

RNN Encoder/Decoder without Attention
Bottleneck



[[Subject:](#) Professor Perry]

Connecting the Encoder and Decoder through the "bottleneck" of $\bar{\mathbf{h}}_{(\bar{T})}$ thus burdens the

- Encoder: passing knowledge forward to the bottleneck
- Decoder: passing knowledge from the bottleneck
 - to all subsequent positions (red box within each circle)

This results in an inefficient use of the model's latent state variable

- In addition to
 - the Encoder and Decoder allocating some of the model's latent state for "control"
 - guiding the loop that processes the Input, or generates the output positions in the template
- It must **also** allocate some of the model's latent state for "knowledge storage"
 - in order to Lookup the concrete value corresponding to a place-holder in the Output template

Cross Attention

Reference

[Neural Machine Translation by Jointly Learning To Align and Translate \(https://arxiv.org/pdf/1409.0473.pdf\)](https://arxiv.org/pdf/1409.0473.pdf) paper that introduced Attention
(<https://arxiv.org/pdf/1409.0473.pdf>).

The flaw in the Encoder-Decoder without Attention is

- the input $\mathbf{x}$ is processed *only once*
- by the Encoder
- which has to summarize it in $\bar{\mathbf{h}}_{(\bar{T})}$

We will introduce a mechanism called *Attention*

- that allows the input sequence to be *re-visited* at each time step of Output generation

This will result in a cleaner separation between control memory and input memory

Attention allows the Decoder

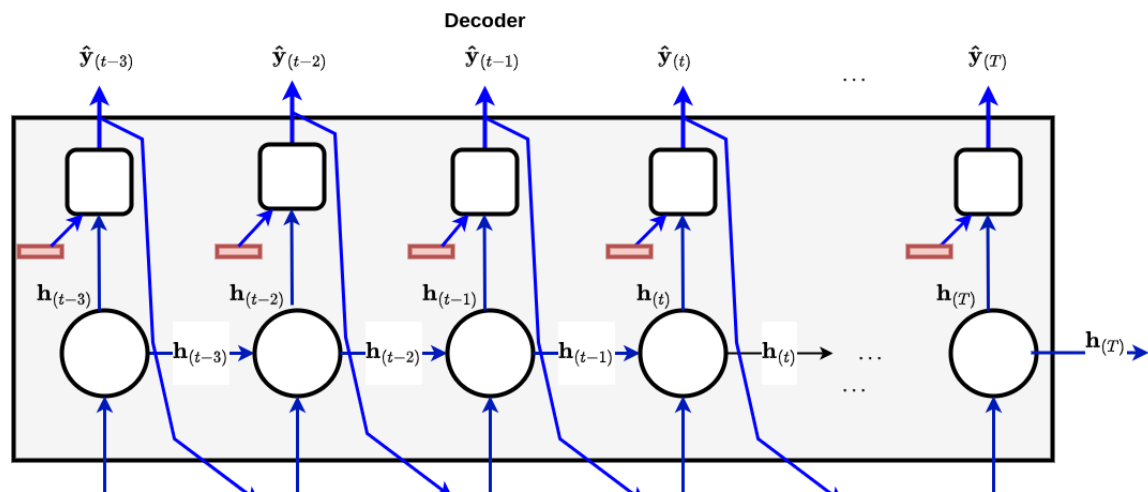
- to directly access all of the Encoder latent states $\bar{\mathbf{h}}_{(1)} \dots \mathbf{h}_{(\bar{T})}$
- at each time step of the Decoder

Thus, there is no need

- for an Encoder to create a full dictionary as the final Encoder latent state $\bar{\mathbf{h}}_{(\bar{T})}$
- for the Decoder to keep the dictionary in all it's latent states $\mathbf{h}_{(1)} \dots \mathbf{h}_{(T)}$

Here is a picture of an Encoder/Decoder augmented with Attention

- we have add an additional box to the diagram for the NN that implements the function D
 - that maps $\mathbf{h}_{tp}$ to $\mathbf{y}_{tp}$



This is one "flavor" of Attention known as *Cross Attention*

- one component (the Decoder) uses as input (*attends to*) the output of another component (the Encoder)

Decoder has *direct access* to **all** outputs (i.e., Latent states) of the Encoder

- each Encoder output is proxy for a prefix of the input

The pink box is the sequence of Encoder outputs

$$\bar{\mathbf{h}}_{(1:\bar{T})}$$

Notice that the final Encoder latent state $\bar{\mathbf{h}}_{(T)}$ is **no longer** connected to the Decoder.

What is going on inside the "box" implementing function D that we added at each time step?

The box's input at step

- the Decoder latent state $\mathbf{h}_{tp}$
- the collection of Encoder latent states $\bar{\mathbf{h}}_{(1)} \dots \mathbf{h}_{(T)}$
 - the red box in the above diagram

That is, it is computing a $\hat{\mathbf{y}}_{tp}$ that is a function of both $\mathbf{h}_{tp}$ and $\bar{\mathbf{h}}_{(1)} \dots \mathbf{h}_{(T)}$

$$\hat{\mathbf{y}}_{tp} = D(\mathbf{h}_{tp}, [\bar{\mathbf{h}}_{(1)} \dots \mathbf{h}_{(T)}])$$

Self-Attention: removing the Encoder loop

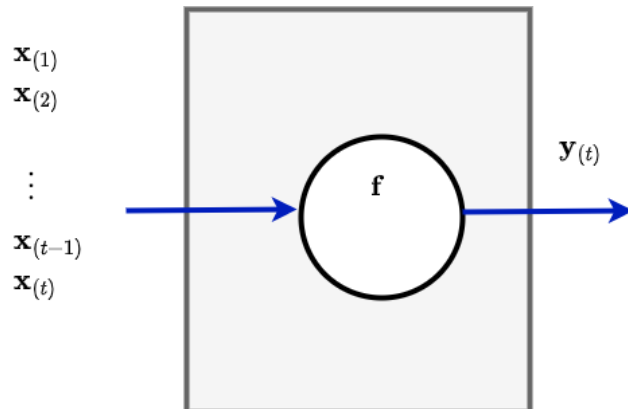
There is an alternative to the loop architecture for processing sequences

- the direct function approach

The alternative to the loop was to create a "direct function"

- Taking a **sequence** $\mathbf{x}_{(1\dots)}$ as input
- Outputting $\hat{\mathbf{y}}_{tp}$

Direct function



Note that output at position *no longer depend* on latent state from position (-1)

- we have removed the loop !

Thus, the Encoder can output *all* elements of sequence $\backslash \bar{\mathbf{h}}$ *simultaneously*

- n.b. the Encoder output sequence will still be denoted $\backslash \bar{\mathbf{h}}_{(1:\bar{T})}$

even though the outputs are no longer *latent states*

- each output position is independent of previous output
- only dependent on input

We removed the "loop" architecture of the Encoder by using the direct function approach

- the mechanism enabling each position of the Encoder output to *attend* to the entire sequence x is called *Self-Attention*
 - Notice: no dependency arrow between circles in the Encoder
- Encoder output is a direct function of **all** positions in the input
 - all Encoder output positions can be computed *in parallel*

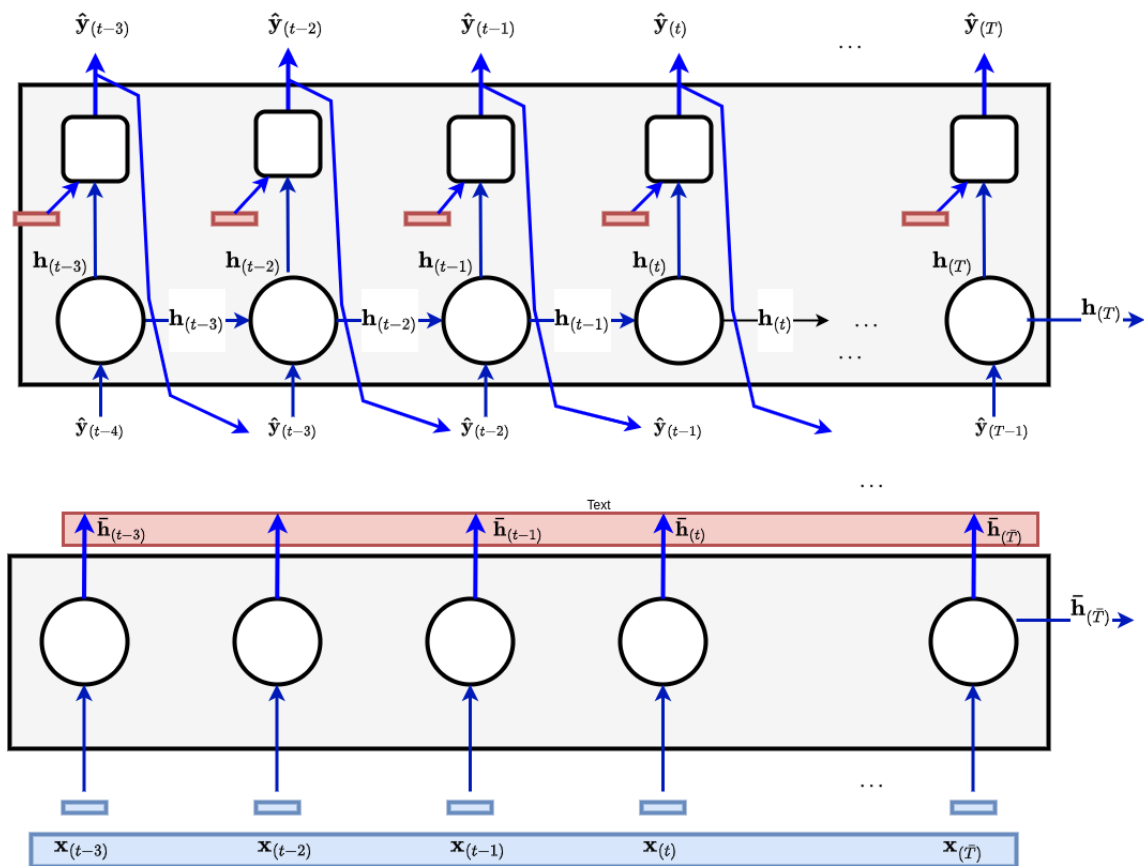
The blue box represents the *entire* input sequence

$\backslash \mathbf{x}_{(1:\bar{T})}$

We no longer refer to the Encoder output as a Latent state

- no more loop !

RNN Encoder/Decoder with Cross Attention/Decoder Self Attention



Masked Self Attention

With *unmasked* Self Attention

- Encoder output $\bar{\mathbf{h}}_{tp}$ at position
- is a function of **all** inputs $\mathbf{x}_{(1:\bar{T})}$
 - including positions after

This is useful, for example, when the meaning of a word depends on its *entire* context.

- as in our motivating example

For certain tasks (not so for our motivating example), full visibility of all inputs is not permissible

- "looking into the future"
 - e.g., predict stock return based only on **past** information

In this case, we use *masked* Self Attention

- we use a mask to hide inputs from position onwards so that
- output $\bar{\mathbf{h}}_{tp}$ at position
- is a function only of **preceding** inputs $\mathbf{x}_{(1:-1)}$

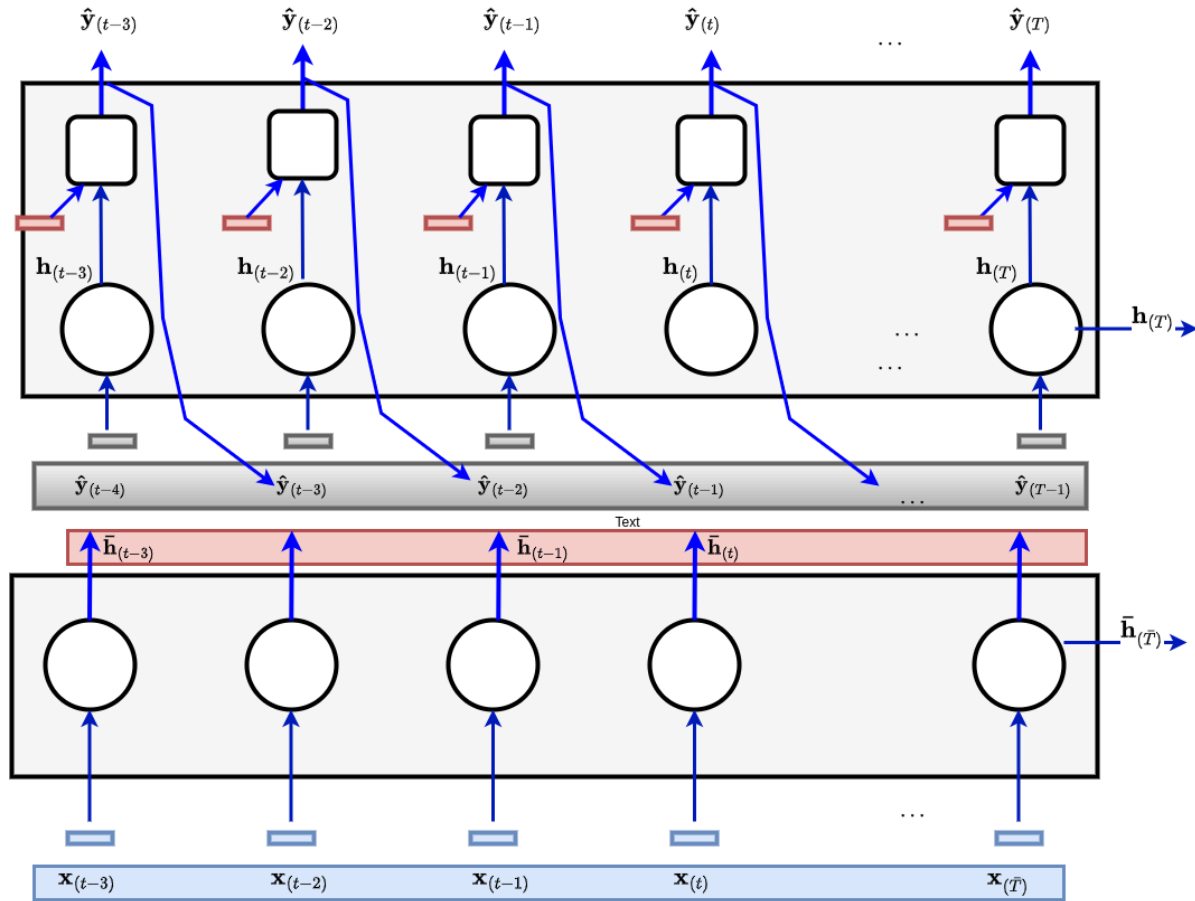
We will see the use of masking in the next section.

Causal Masked Self Attention: removing the Decoder loop

Finally we remove the loop architecture for the Decoder as well using a different "flavor" of Self-Attention

- Masked Self-Attention.

RNN Encoder/Decoder with Cross Attention and Self Attention (Encoder/Decoder)



The grey box represents the *entire* output sequence

$$\hat{\mathbf{y}}_{(1:T)}$$

From this diagram: it appears that

- the Encoder/Decoder can produce output $\hat{\mathbf{y}}_{tp}$
- while attending to outputs *that have not yet been generated* at the start of step
- "looking into the future"

$$\hat{\mathbf{y}}_{(:T)}$$

That is, it is computing

$$\text{prc} \hat{\mathbf{y}}_{tp} \hat{\mathbf{y}}_{(1:T)}$$

What is going on ?

Teacher forcing at training time

The *Auto-Regressive* Decoder behavior

- constructs output sequence $\hat{\mathbf{y}}$ sequentially
 - output $\hat{\mathbf{y}}_{t_p}$ of step
 - becomes part of the input $\hat{\mathbf{y}}_{(1:t)}$ of step $t+1$

So how can the entire output sequence $\hat{\mathbf{y}}_{(1:T)}$ be available before final step T ?

We need to distinguish between behavior

- during *training*
- versus during *inference*

During inference, clearly we can only compute

$$\text{prc}(\hat{\mathbf{y}}_{t_p}, \hat{\mathbf{y}}_{(1:-1)})$$

because we haven't generated the future outputs yet.

But at training time, example is

$$\langle \backslash \mathbf{y}_{(1:-1)}, \backslash \mathbf{y}_{\backslash \mathbf{tp}} \rangle$$

We predict *only the immediate next* target $\backslash \mathbf{y}_{\backslash \mathbf{tp}}$

- *not* the full suffix $\hat{\backslash \mathbf{y}}_{(:\mathbf{T})}$

The prediction is conditioned on the *true* target prefix $\backslash \mathbf{y}_{(1:-1)}$

- *not* the prefix generated during training $\hat{\backslash \mathbf{y}}_{(1:-1)}$

The Auto-Regressive behavior is eliminated during training !

Training in this manner has a big advantage.

In a perfect world, when predicting $\hat{\mathbf{y}}_{\text{tp}}$

- the Auto-Regressive behavior during training would result in
- the prefix $\hat{\mathbf{y}}_{(1:-1)}$ of the generated output
- matching the true target

$$\hat{\mathbf{y}}_{(1:-1)} = \mathbf{y}_{(1:-1)}$$

But

- if any element of the *generated* prefix is wrong
$$\hat{y}_{(')} \neq y_{(')} \text{ for } ' <$$
- it is likely that all *subsequent predicted outputs* $\hat{y}_{('+1:T)}$ will be wrong
- because each subsequent output **is conditioned on incorrect** $\hat{y}_{(')}$

That is

- a single mis-predicted element of the sequence
- causes a catastrophic chain of errors

Training a model under such conditions would be difficult

- the incorrect sequences don't even come from the true distribution of inputs !
- violating the Fundamental Theorem of Machine Learning

To avoid this, our training examples compute

$$\text{prc} \backslash \hat{\mathbf{y}}_{\text{tp}} \backslash \mathbf{y}_{(1:-1)}$$

rather than

$$\text{prc} \backslash \hat{\mathbf{y}}_{\text{tp}} \backslash \hat{\mathbf{y}}_{(1:-1)}$$

That is: we train on *target* prefixes rather than train-time generated prefixes.

This is called *Teacher Forcing*.

Teacher forcing can be implemented

- by making the entire target output sequence

$\backslash \mathbf{y}_{(1:T)}$

i.e., the grey box in the diagram

- available at training time via setting example to

$\langle \backslash \mathbf{y}_{(1:T)}, \backslash \mathbf{y}_{\backslash t_p} \rangle$

- and using *Causal masking*
- to make only prefix $\backslash \mathbf{y}_{(1:-1)}$ visible during the prediction of $\backslash \mathbf{y}_{\backslash t_p}$

Masked attention

Hopefully it is clear that, regardless of whether we are computing $\hat{y}_{tp}$

- at training time
- at inference time

the computation should depend only on positions $1 : -1$ of the output.

- can't peek into the future

To enforce this

- we *mask* the outputs
- so that only positions $1 : -1$ are visible when generating output position

The general mechanism of hiding some inputs is called

- **Masked Self-Attention**

The specific masking of only future positions is called

- **Causal Masked Self-Attention or Causal Self-Attention**

Visualizing Attention

We can illustrate the behavior of Neural Networks that have been augmented with Attention through diagrams.

- at a particular output position
- we can display the amount of "attention"
- that each position in the input receives

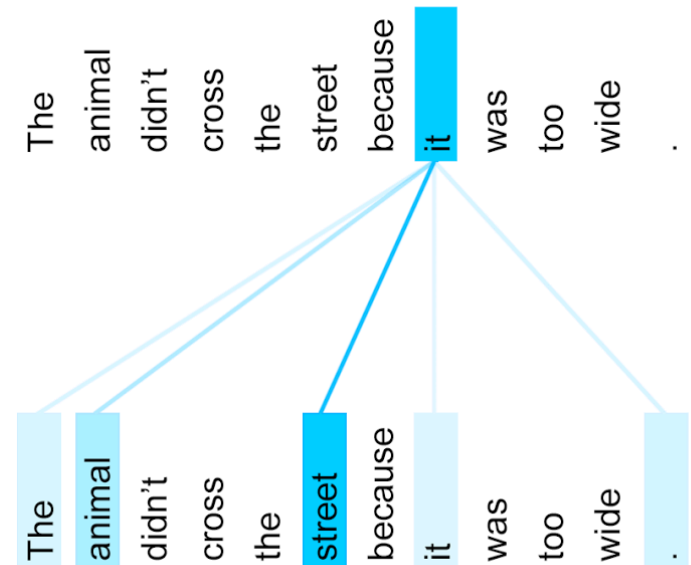
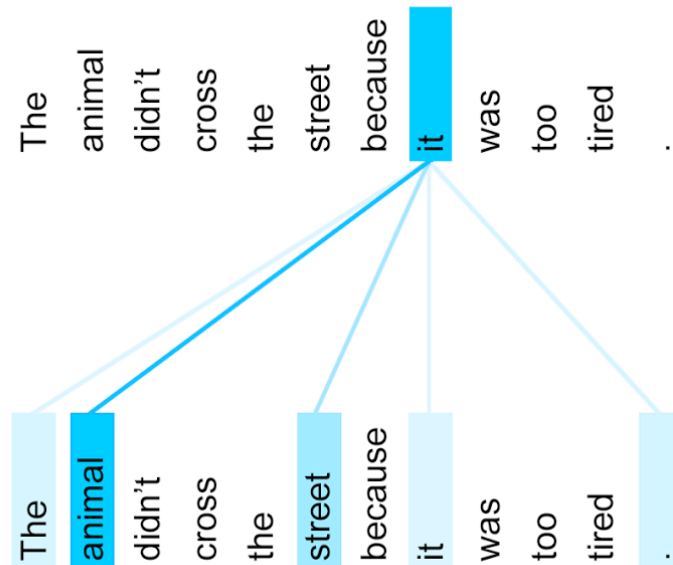
Visualizing Self-Attention

Self Attention can be used to create a Context Sensitive Encoding of words

- The meaning of a word may change depending on the rest of the sentence

We can illustrate this with an example: how the meaning of the word "it" changes

- The thickness of the blue line indicates the attention weight that is given in processing the word "it".



Much of the recent advances in NLP may be attributed to these improved, context sensitive embeddings.

We note that simple Word Embeddings

- also capture "meaning"
- but are *not* sensitive to context

Visualizing Cross Attention

The Entailment task

The *Entailment* task is a binary classification task based on two sentence

- first sentence (the *premise*)
- second sentence (the *hypothesis*)

Classify: Does the Hypothesis logically follow (is *entailed* by) the Premise ?

Here is an illustration of what part of the Premise is attended to as we encounter the Hypothesis

Attention: Entailment

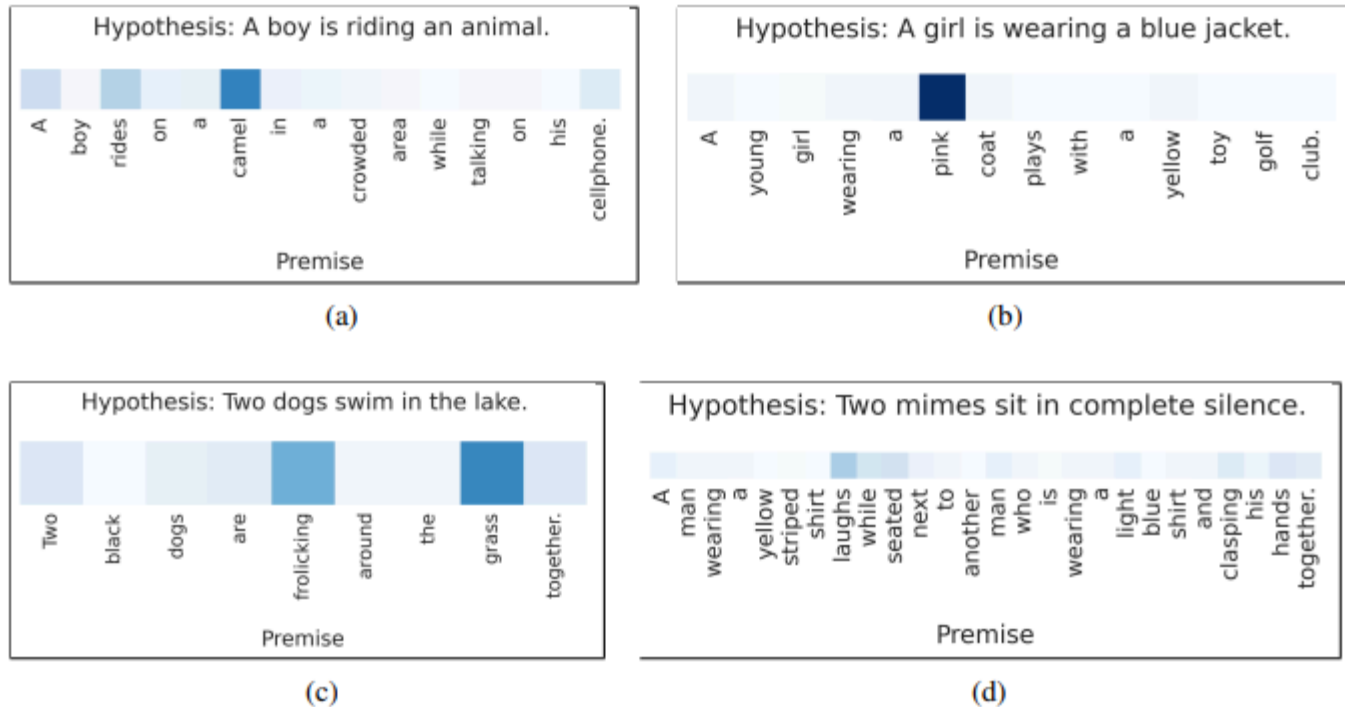


Figure 2: Attention visualizations.

Attribution: <https://arxiv.org/pdf/1509.06664.pdf#page=6>
(<https://arxiv.org/pdf/1509.06664.pdf#page=6>)."

The Date Normalization task

- Source: Dates in free-form: "Saturday 09 May 2018"
- Target: Dates in normalized form: "2018-05-09"

[link \(https://github.com/datalogue/keras-attention#example-visualizations\)](https://github.com/datalogue/keras-attention#example-visualizations)

The Image captioning task

- Source: Image
- Target: Caption: "A woman is throwing a **frisbee** in a park."
- Attending over *pixels* **not** sequence

Visual attention



A woman is throwing a **frisbee** in a park.

Attribution: <https://arxiv.org/pdf/1502.03044.pdf> (<https://arxiv.org/pdf/1502.03044.pdf>)

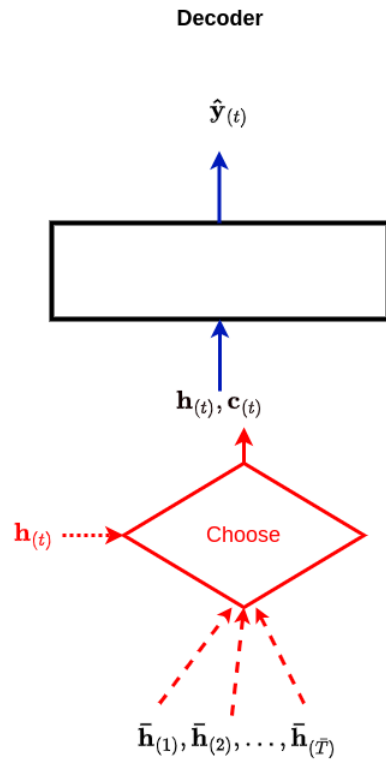
Implementing Attention (Preview): Inside the NN for D

Recall the Decoder conditions output $\hat{\mathbf{y}}_{tp}$ on

- Decoder state $\mathbf{h}_{tp}$
- Encoder output sequence $\bar{\mathbf{h}}_{(1:\bar{T})}$

Here is a high-level view of the inner workings of the NN for D :

Decoder output transformation with attention



Inside the box:

- the Decoder latent state $\mathbf{h}_{tp}$ is used as a *query*
- which is matched against each of the Encoder outputs
- resulting in one Encoder output being chosen as $\mathbf{c}_{tp}$

The chosen Encoder output $\mathbf{c}_{tp}$ and Decoder latent state $\mathbf{h}_{tp}$

- are input to another Neural Network
- which produces output $\hat{\mathbf{y}}_{tp}$

The "Choose" box implements an *Attention* mechanism, which allows the Decoder

- to **attend to** the part of Input $\mathbf{x}$ (represented via some Encoder latent state $\bar{\mathbf{h}}_{(-)}$)
- that is *relevant* for producing $\hat{\mathbf{y}}_{tp}$
- exactly when it is needed

This seems very natural to a human

- rather than memorizing details (e.g., the big dictionary $\bar{\mathbf{h}}_{(\bar{T})}$ in the architecture without Attention)
- we refer back to the context
- focusing of only the part that is immediately needed

The discussion of the **implementation** of Attention will be deferred to a later module [Attention lookup \(Attention Lookup.ipynb\)](#).

For now, think of the "Choose" box as a Context Sensitive Memory (as described in the module on [Neural Programming \(Neural Programming.ipynb#Soft-Lookup\)](#))

- Like a Python dict
 - Collection of key/value pairs: $\langle \bar{\mathbf{h}}_{(-)}, \bar{\mathbf{h}}_{(-)} \rangle$
 - Key is equal to value; they are latent states of the Encoder
- But with *soft* lookup
 - The current Decoder state $\mathbf{h}_{tp}$ is presented to the CSM
 - Called the *query*
 - Is matched across each key of the dict (i.e., a latent state $\bar{\mathbf{h}}_{(-)}$)
 - The CSM returns an approximate match of the query to a *key* of the dict
 - The distance between the query and each key in the CSM is computed
 - The Soft Lookup returns a *weighted* (by inverse distance) sum of the *values* in the CSM dict

Multi-head attention: two heads are better than one

Perhaps when generating the output for position of the output sequence

- we need to attend to *more than one* position of the sequence being attended to
 - need to know both gender and plurality of subject
- that is: we want an Attention layer to output multiple items.

We can attend to n positions

- by creating n separate Attention mechanisms
- each one called a *head*

This behavior is referred to as *Multi-head attention*

This type of behavior is common to many layer types in a Neural Network

- a Dense layer l may produce a vector $\mathbf{y}_l$ where $n_l > 1$
- a Convolutional layer l may produce outputs (for each spatial location) for many channels

We have referred to this as layer $\ll$ producing n_l features.

It would be natural for an Attention layer to output many "features" to enable attention to many positions.

In practice, this is sometimes (always ?) not done

- Model architectures (e.g., the Transformer) are simplified when the inputs/outputs of each sub-component
- have the same length
- often denoted as d or d_{model} in the Transformer

When a Transformer needs to attend to n positions

- it uses n Attention heads
- each outputting a vector of length $\frac{d}{n}$
- which are concatenated together to produce a single output of length d

In essence

- we create n "mini-attention" heads
 - we use queries
 - and input sequences -which are a *fraction* of the original lengths
- the outputs of the n mini-heads can be combined into an output of length d

So multi-head attention is compatible (in terms of shape of input and output) with single head attention.

When we have n heads

- Rather than having one Attention head operating on vectors of length d
 - producing an output of length d (weighted sum of values in the CSM)
- We create n Attention heads operating on vectors (keys, values, queries) of length $\frac{d}{n}$.
 - Output of these smaller heads are values, and hence also of length $\frac{d}{n}$
- The final output concatenates these n outputs into a single output of length d
 - identical in length to the single head
- we project each of these length d vector into vectors of length $\frac{d}{n}$

The picture shows n Attention heads.

Note that each head is working on vectors of length $\frac{d}{n}$ rather than original dimensions d .

- variables with superscript (j) are of fractional length

Details are deferred to the module [Attention lookup \(Attention Lookup.ipynb\)](#).

Each head j uniquely transforms the query $\mathbf{h}_{\text{tp}}$ and the key/value pairs $\bar{\mathbf{h}}_{(1)} \dots \bar{\mathbf{h}}_{(T)}$ being queried.

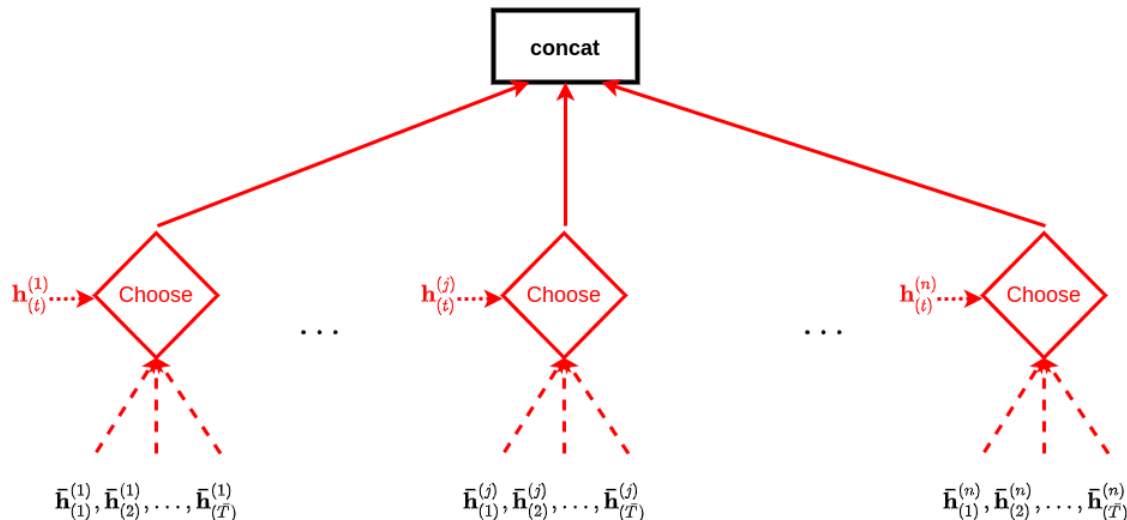
- into $\mathbf{h}_{\text{tp}}^{(j)}$ and the key/value pairs $\bar{\mathbf{h}}_{(1)}^{(j)} \dots \bar{\mathbf{h}}_{(T)}^{(j)}$
- all vectors/vector elements with superscripts are of length $\frac{d}{n}$ rather than the original length d

Decoder Multi-head Attention

Per-head query and value

$$\mathbf{h}_{(t)}^{(j)} = \mathbf{W}_{\text{query}}^{(j)} \mathbf{h}_{(t)}$$

$$\bar{\mathbf{h}}_{(t)}^{(j)} = \mathbf{W}_{\text{value}}^{(j)} \bar{\mathbf{h}}_{(t)}$$



Transformers (preview)

Using various flavor of Attention

- we have replaced the Encoder and Decoder RNN's in an Encoder/Decoder architecture
- with a new layer type
 - direct function approach to sequences

This new architecture is the basis for the *Transformer* layer

- a key advance in modern Deep Learning, particularly for NLP
- which we will study in more depth in a subsequent module

Conclusion

We recognized that the Decoder function responsible for generating Decoder output $\hat{\mathbf{y}}_{tp}$

$$\hat{\mathbf{y}}_{tp} = D(\mathbf{h}_{tp}; \mathbf{s})$$

was quite rigid when it ignored argument $\mathbf{s}$.

This rigidity forced Decoder latent state $\mathbf{h}_{tp}$ to assume the additional responsibility of including Encoder context.

Attention was presented as a way to obtain Encoder context through argument $\mathbf{s}$.

In [2]: `print("Done")`

Done

